Rapport backend — AZȮ

Ce qui a été fait sur l'API (`azo-backend`) — pour Dorin

20 octobre 2026 · 3 commits de code poussés sur main (`77fa4e64`, `ede1c603`, `8f00c7d9`) · 33 suites / 379 tests verts

1. En résumé

Le **commercialage** : tables, routes administrateur et commercial, rattachement des prestataires à l'inscription. `tsc` sans erreur. Une migration (une table, deux colonnes).

> **Déploiement :** le push de `main` déclenche le déploiement sur Render ; `/health` doit afficher `77fa4e64` ou plus récent. Le serveur doit être déployé avant l'APK.

2. Données (migration `20261018100000_referrals`)

- Table `Referrer` : `userId` (unique, le compte du commercial), `code` (unique, « AZO- » + 6 caractères sans I, L, O, 0, 1), `active`, `note`, `createdById` (administrateur), `createdAt`.
- `User.referrerId` (clé étrangère vers `Referrer`, `ON DELETE SET NULL`, indexée) et `User.referredAt` : le commercial d'un prestataire. Le rôle du commercial reste `CLIENT` ; on le reconnaît à sa ligne `Referrer` (pas de nouvelle valeur de l'énumération `Role`).
- Appliquée sur une base vide ; aucun écart entre le schéma et les migrations pour ces objets.

2bis. Code promo « COM-AZO001 » et lien (commit `ede1c603`)

- `Referrer.code` : **`COM-AZO` + numéro d'ordre sur au moins 3 chiffres** (le plus grand numéro déjà attribué + 1 ; la contrainte d'unicité protège d'une création simultanée). `normalizeReferralCode` accepte « com azo 1 », « comazo001 », « COM-AZO001 » ; les anciens codes `AZO-XXXXXX` (6 caractères sans I, L, O, 0, 1) restent acceptés.
- **Lien personnel** renvoyé avec le commercial (`link` : création, liste, fiche, `GET /referrals/me`) : `https://azo.angidsnpro.com/?ref=COM-AZO001#telecharger`. L'adresse de base se règle par la variable d'environnement facultative `REFERRAL_LINK_BASE`.
- `GET /referrals/me` renvoie aussi le numéro de téléphone du commercial. Les messages d'erreur parlent de « commercial ». Le nom des tables et colonnes (`Referrer`, `referrerId`) et des routes (`/admin/referrals`, `/referrals/me`) est inchangé : pas de nouvelle migration.
- Vérifié sur une base jetable (7 vérifications) : COM-AZO001 puis COM-AZO002, numérotation qui continue après un COM-AZO007 créé à la main (COM-AZO008), lien correct, connexion du commercial, liste et fiche d'administrateur avec code et lien, rattachement avec « com azo 1 » / « comazo002 » / « COM-AZO002 », code inconnu signalé, recrue visible avec numéro masqué. Tests ajoutés dans `referrals.spec.ts` (12 au total).

3. Routes

Route	Rôle
`POST /admin/referrals` {fullName, phone, note?}	Admin : crée le commercial (compte + code). 409 si le numéro est déjà commercial, client inscrit avec identité, prestataire ou membre d'agence.
`GET /admin/referrals`	Admin : commerciaux avec leurs chiffres par catégorie (100 derniers).
`GET /admin/referrals/:id`	Admin : fiche et liste complète des recrues (numéros complets).
`PATCH /admin/referrals/:id` {active?, note?}	Admin : désactiver / réactiver, note.
`POST /admin/referrals/:id/attach` {phone}	Admin : rattache un prestataire existant (404 inconnu, 400 simple client, 409 déjà commercialé ou commercial).
`DELETE /admin/referrals/referral/:userId`	Admin : retire un recrue de son commercial.
`GET /referrals/me`	Commercial : ses chiffres et ses recrues (numéros partiellement masqués). 403 pour un compte qui n'est pas commercial ou un commercial désactivé.

- **Connexion** : `POST /auth/verify-otp` accepte `referralCode` (facultatif) et renvoie `referralStatus` (`APPLIED` , `INVALID` ou `null`) et `referrer` ({code, active} si le compte est un commercial : l'application ouvre alors l'espace commercial sans formulaire d'identité). Le code n'est pris en compte que pour une inscription Speeder, coursier, livreur ou agence, sur un compte sans commercial et sans identité client ; un code inconnu, désactivé ou celui du prestataire lui-même est signalé sans bloquer l'inscription.

4. Calcul du suivi (`referrals/referrals.module.ts` , `referral-logic.ts`)

- **Catégorie** : Speeder (`DRIVER`), agence, coursier personnel (`COURIER` + `COURSIER_PERSONNEL`), livreur express (autres `COURIER`).
- **État** : dossier validé + au moins une course ou livraison terminée ces **30 jours** = actif ; validé sans activité récente = inactif ; brouillon, déposé, en examen, information demandée = en attente ; refusé ; suspendu.
- **Activité** : courses terminées (`Ride.completedAt`) et livraisons terminées (`Delivery.createdAt` , faute d'heure de fin). Une agence compte l'activité de sa flotte (ses chauffeurs rattachés, sans son propriétaire).
- **Recharges** : transactions `kind = RECHARGE` et `status = CONFIRMED` du portefeuille du recrue (nombre, total, dernière date) ; les recharges en attente ne comptent pas.
- Requêtes groupées (pas de requête par recrue).

5. Vérifications

- **Essai de bout en bout** sur une base jetable avec l'API : **22 vérifications sur 22** (création du commercial, refus de doublons et de comptes déjà inscrits, connexion du commercial, 403 pour un client, inscription avec code en majuscules, minuscules ou avec espace, code invalide signalé, client jamais rattaché, rattachement et refus côté administrateur, chiffres par catégorie, actifs / inactifs / en attente, recharges 7 000 F sans la recharge en attente, missions et flotte, numéros masqués côté commercial et complets côté administrateur, retrait et nouveau rattachement, désactivation).

- **Tests automatiques** (`test/referrals.spec.ts` , 11 tests) : codes, normalisation, catégories, états actif / inactif, masquage, rattachement par code, création par l'administrateur, accès à l'espace commercial.
- **Non vérifié** : écrans sur téléphone.

6. Limites et suites possibles

- Pas de récompense ni de commission pour les commerciaux : suivi seulement.
- La liste des commerciaux calcule les chiffres commercial par commercial (suffisant pour quelques dizaines de commerciaux).
- Le numéro d'un commercial doit être neuf : un client inscrit ou un prestataire ne peut pas le devenir.

7. Connexion administrateur par mot de passe (nouveau)

- **Faille corrigée** : tant que `SMS_PROVIDER` valait « simulation », le code 0000 ouvrait n'importe quel compte, administrateur compris. `verifyOtp` refuse désormais le rôle ADMIN (« Les comptes administrateur se connectent avec leur mot de passe sur la console web »).
- **Route** `POST /auth/admin-login` `{phone, password}` : jeton de 12 h, même message d'erreur pour un numéro inconnu, un mauvais mot de passe ou un compte non administrateur, verrouillage après 5 échecs en 15 minutes (en mémoire, par numéro et par adresse), compte bloqué refusé.
- **Mot de passe** : haché avec scrypt (`scrypt$16384$sel$hash`), colonne `User.passwordHash` (migration `20261020120000_admin_password` , une colonne ajoutée). `scripts/create-admin.js` lit `ADMIN_PASSWORD` dans l'environnement (10 caractères minimum, lettre et chiffre) et refuse la base distante sans `--allow-remote` .
- **CORS** : `main.ts` autorise uniquement les adresses listées dans la variable `CORS_ORIGINS` (séparées par des virgules). Vérifié après déploiement : l'en-tête `access-control-allow-origin` renvoie l'adresse de la console.
- **Tests** : `test/admin-login.spec.ts` (9 tests : hachage, compatibilité script/serveur, force du mot de passe, succès, messages identiques, verrouillage, compte bloqué, numéro invalide, refus de l'administrateur avec le code 0000). Suite complète : **379 tests verts**, `tsc` sans erreur.
- **Production** : le serveur répond à `/auth/admin-login` (essai : « Numéro ou mot de passe incorrect »). **Reste à faire par vous** : créer le mot de passe du patron sur la vraie base avec `create-admin` . L'adresse de la base a circulé dans une conversation : **changer le mot de passe de la base** sur Render et mettre à jour `DATABASE_URL` du service.

8. Commits

Commit	Contenu
`8f00c7d9`	Connexion administrateur par mot de passe (migration `20261020120000`), CORS configurable, console web
`ede1c603`	Commerciaux : code COM-AZO001, lien personnel, messages
`77fa4e64`	Commerciaux (commercialage) : migration `20261018100000` , routes administrateur et commercial, code à l'inscription